

K R O G E R

Sourcing Category Recommendation

End-to-End Agentic Flow

Technical Deep-Dive Document

Version: 1.0

Status: Draft — Internal Review

Date: August 2026

Scope: Frontend · Backend · AI Agent · Databricks · Security · Observability

Classification: Internal — Confidential

Table of Contents

1. Purpose and Scope

This document provides a detailed technical walkthrough of the end-to-end agentic flow for the Kroger Sourcing Category Recommendation system. Unlike the broader Technical Design Document, this paper focuses exclusively on the AI-powered chat path — tracing a single user request from the moment it leaves the browser through every layer of the system and back, with full detail on authentication, data formats, protocols, security gates, and observability instrumentation at each hop.

The intended audience is the engineering team building and reviewing each layer: frontend engineers, backend Python engineers, AI/ML engineers working on the LangGraph agent, data engineers owning the Databricks integration, and security engineers reviewing OAuth flows and AI safety controls.

1.1 What This Document Covers

- The complete agentic request lifecycle from browser keystroke to streamed response
- Frontend: CopilotKit/AG-UI setup, SSE POST mechanics, token rendering, auth flow
- Backend: FastAPI route, AG-UI runner, SSE relay, per-route security and timeout configuration
- AI Agent: LangGraph graph structure, two-agent A2A design, tool execution, Postgres checkpointing
- Databricks: MCP protocol, tool call construction, Unity Catalog access control, query sanitisation
- AI Gateway: APIM AI routing, model selection, token quota enforcement, semantic caching
- Security: OAuth2 flow at every hop, token propagation, Managed Identity, Prompt Shields
- Observability: LangSmith trace structure, Azure Monitor log sources, alert wiring

2. Architecture Overview — The Agentic Path

The agentic path is activated when a user submits a message in the chat interface. It is fundamentally different from the plain REST data-fetch path (invoices, suppliers, opportunities) in every dimension: protocol (SSE vs JSON), timeout (300s vs 30s), workloads involved (all three vs Workload 2 only), and cost profile (LLM tokens + Databricks compute vs fast DB read).

2.1 Component Map

Layer	Component	Technology	Role in Agentic Path
Client	Browser	React 19, CopilotKit	Renders chat UI, sends POST, consumes SSE stream
Client → Backend	App Gateway + APIM	Azure WAF + APIM Premium	TLS termination, WAF, OIDC validation, SSE policy, quota
Workload 1	Next.js API route	Next.js 14, /api/copilotkit/route	Server-side SSE relay, session validation
Workload 2	FastAPI + AG-UI Runner	FastAPI, Python, AG-UI middleware	SSE orchestration, invokes Workload 3
Workload 3	LangGraph Orchestrator	LangGraph, Python	Outer-loop agent, state management, delegates to Planner
Workload 3	Semantic Query Planner	LangGraph subgraph / A2A agent	Intent resolution, tool selection, guardrail validation
Workload 3	Tool layer	MCP client, sanitize.py, query.py	Executes governed tool calls against Databricks MCP
AI Safety	Azure AI Content Safety	Prompt Shields API	Prompt inspection (steps 1-2) and response inspection (steps 5-6)
AI Inference	AI Gateway → Foundry LLM	APIM AI, Azure OpenAI / Claude	Routes LLM call, enforces token quota, semantic cache
Data	Databricks MCP	Databricks Managed MCP over Private Link	Executes SQL against Unity Catalog governed tables
State	PostgreSQL Flexible Server	Azure PostgreSQL HA	LangGraph checkpoint persistence across turns
Observability	LangSmith + Azure Monitor	LangSmith, OTel, Log Analytics	Full trace attribution, cost, safety, quality metrics

2.2 High-Level Sequence

The numbered sequence below maps directly to the diagram in the architecture PDF and the detailed sections that follow:

Step 1 User submits chat message

Browser: fetch() POST /api/chat with JSON body; Content-Type: application/json; OIDC token in Authorization header

Step 2 App Gateway + APIM

WAF inspects; APIM validates OIDC token, applies /api/chat policy (300s timeout, buffering OFF), forwards to AKS internal LB

Step 3 Workload 2 — FastAPI + AG-UI Runner

Route handler invokes AG-UI runner; response headers set to text/event-stream; SSE connection opened to browser

Step 4 Workload 2 → Workload 3

AG-UI runner calls LangGraph agent endpoint over internal ClusterIP; async invocation

Step 5 Content Safety — prompt inspection

Agent calls Prompt Shields on user message; if flagged, run terminates; if clean, sanitised prompt continues

Step 6 LangGraph Orchestrator → Semantic Query Planner

Orchestrator delegates to Planner via A2A (JSON-RPC 2.0); Planner resolves intent, selects tool, validates params

Step 7 AI Gateway → LLM inference

Agent calls APIM AI gateway with sanitised prompt; APIM routes to Foundry model; response streamed back

Step 8 Tool execution

Agent executes planned tool call (query_spend_table / find_supplier / query_semantic_planner_rag) via Databricks MCP Private Link

Step 9 Content Safety — response inspection

Agent calls Prompt Shields on LLM+tool combined response; if clean, response proceeds to stream

Step 10 SSE stream back to browser

Response chunks flow back through Workload 2 AG-UI runner → APIM → App Gateway → browser; tokens rendered incrementally

3. Frontend — Workload 1 (Next.js + CopilotKit)

The frontend is a Next.js 14 application using React 19. The chat interface is built on CopilotKit, which implements the AG-UI (Agent User Interaction) protocol — an open, event-based protocol for streaming agent state to user interfaces over Server-Sent Events.

3.1 CopilotKit Setup

The CopilotKit provider wraps the entire application, establishing the connection context for all AI hooks:

```
// app/layout.tsx
import { CopilotKit } from "@copilotkit/react-core";

export default function RootLayout({ children }) {
  return (
    <CopilotKit runtimeUrl="/api/copilotkit/route">
      {children}
    </CopilotKit>
  );
}
```

The runtimeUrl points to the Next.js API route (/api/copilotkit/route) which acts as a server-side relay to the FastAPI BFF. This is intentional: the browser never directly calls the BFF — it calls Next.js, which adds server-side auth validation before forwarding.

3.2 Chat Hooks

```
// components/ChatPanel.tsx
import { useCopilotChat, useCopilotReadable } from "@copilotkit/react-core";
import { MessageRole, TextMessage } from "@copilotkit/runtime-client-gql";

export function ChatPanel({ invoices, suppliers }) {
  // Make structured data readable by the agent
  useCopilotReadable({
    description: "Current invoice and supplier context for this session",
    value: { invoices, suppliers },
  });

  const { visibleMessages, appendMessage, isLoading } = useCopilotChat();

  const handleSend = (userText) => {
    appendMessage(new TextMessage({ content: userText, role: MessageRole.User }));
  };
}
```

useCopilotReadable passes structured invoice/supplier data as readable context to the agent — meaning the agent can see which records the user is currently viewing when formulating recommendations. This reduces redundant tool calls for data that is already on screen.

3.3 Next.js API Route — Server-Side Relay

The `/api/copilotkit/route` Next.js API route bridges CopilotKit's client protocol to the FastAPI BFF. It performs server-side OIDC session validation before forwarding — providing a second validation layer beyond APIM.

```
// app/api/copilotkit/route.ts
import { CopilotRuntime, copilotRuntimeNextJSAppRouterEndpoint } from
"@copilotkit/runtime";
import { getSession } from "next-auth";

const runtime = new CopilotRuntime({
  remoteEndpoints: [{
    url: process.env.BFF_CHAT_URL, // sourced from Key Vault at build time
  }],
});

export const POST = async (req) => {
  const session = await getSession();
  if (!session) return new Response("Unauthorized", { status: 401 });

  const { handleRequest } = copilotRuntimeNextJSAppRouterEndpoint({
    runtime,
    req,
    endpoint: "/api/copilotkit/route",
  });
  return handleRequest(req);
};
```

3.4 The SSE POST Mechanics

This is the most technically important detail in the frontend. A common misconception is that SSE requires the native EventSource API. EventSource is GET-only and cannot send a JSON body. For the agentic path, the browser makes a POST with a JSON body and receives a streaming SSE response on that same connection.

Property	Value	Explanation
HTTP method	POST	"EventSource" pattern is not used — <code>fetch()</code> with <code>ReadableStream</code> is used instead
Request Content-Type	application/json	User message, session ID, readable context sent as JSON body
Request Authorization	Bearer {oidc_token}	Ping OIDC token; validated by APIM before request reaches BFF
Response Content-Type	text/event-stream	SSE format; browser reads incrementally via <code>ReadableStream</code>
Response Transfer-Encoding	chunked	No Content-Length; connection stays open until agent completes
Response Cache-Control	no-cache	Required for SSE; prevents any intermediate buffering

Property	Value	Explanation
Connection lifecycle	Single HTTP connection	Request open → tokens stream → agent done → connection closed

CopilotKit handles the fetch + ReadableStream plumbing internally. Each chunk is an AG-UI protocol event (text/event-stream formatted). The client parses event types and updates React state accordingly:

```
// AG-UI event types that arrive as SSE chunks:
event: RUN_STARTED           → agent run begun; show loading indicator
event: TEXT_MESSAGE_CHUNK    → partial token; append to message in UI
event: TOOL_CALL_START       → agent calling a tool; show tool indicator
event: TOOL_CALL_END         → tool result received; hide tool indicator
event: STATE_SNAPSHOT        → full agent state update
event: RUN_FINISHED          → stream complete; close connection
event: RUN_ERROR              → agent error; surface to user gracefully
```

3.5 Authentication Flow — Frontend

Authentication is handled via Ping SAML/OIDC. The token is stored in an httpOnly cookie — never in localStorage or sessionStorage, which are accessible to JavaScript and therefore vulnerable to XSS.

Step	Action	Security Note
1	User navigates to app; no session cookie present	Next.js middleware redirects to Ping OIDC /authorize endpoint
2	User authenticates at Ping; Ping issues ID token + access token	SAML assertion or OIDC code flow; Kroger enterprise SSO
3	Next.js callback receives code; exchanges for tokens via server-side fetch	Code exchange never exposed to browser; PKCE enforced
4	Tokens stored in httpOnly Secure SameSite=Strict cookie	JS cannot read this cookie; XSS cannot steal token
5	Every API call includes cookie; Next.js extracts token server-side	Next.js /api/copilotkit/route reads session from cookie, not browser
6	APIM validates Bearer token on every request to BFF	Second validation; pods never validate tokens directly
7	Session timeout: idle > configured period → redirect to Ping	Enforced in Next.js middleware; no silent token refresh for inactive sessions

 **Token storage:** The OIDC access token is stored exclusively in an httpOnly cookie. It is never written to localStorage, sessionStorage, or React state. This prevents token theft via XSS. The token is forwarded to APIM in the Authorization: Bearer header by the Next.js server-side route — the browser never constructs this header directly.

3.6 Content Security Policy

Next.js sets the following CSP headers on all responses to prevent XSS and data exfiltration from the browser:

```
default-src 'self'  
script-src 'self'          - no eval, no unsafe-inline  
connect-src 'self'        - all API calls via same-origin Next.js routes  
only  
img-src      'self' data:   - no external image loading  
frame-ancestors 'none'     - clickjacking prevention  
form-action 'self'        - form submissions same-origin only
```

4. Backend — Workload 2 (FastAPI + AG-UI Runner)

The FastAPI backend (Workload 2) serves two fundamentally different types of traffic: synchronous REST endpoints for structured data fetches, and the agentic chat endpoint that establishes the SSE stream and relays it through to the LangGraph agent. This document focuses on the chat path.

4.1 FastAPI Application Structure

```

app/
  main.py                - FastAPI app, CORS, lifespan, health endpoint
  middleware/
    agui.py              - AG-UI middleware (registered ONLY on /api/chat)
    auth.py              - Workload Identity token helper
  routes/
    chat.py              - POST /api/chat → AG-UI runner → Workload 3
    invoices.py          - GET /api/invoices (plain REST, no SSE)
    suppliers.py         - GET /api/suppliers (plain REST, no SSE)
    opportunities.py     - GET /api/opportunities (plain REST, no SSE)
  backend_service/
    db.py                - Databricks client, Redis client (shared)
    sanitize.py          - input sanitisation, OWASP validation
    query.py             - parameterised SQL builder
    invoices.py          - get_invoices(filters)
    suppliers.py         - get_suppliers(filters)
    opportunities.py     - get_opportunities(filters)

```

4.2 CORS Configuration

CORS is locked to the specific Next.js frontend origin. Wildcard origins (*) are never used — a wildcard would allow any website to make credentialed API calls to the BFF.

```

from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=[ os.environ["FRONTEND_ORIGIN"] ],    # from Key Vault
    allow_credentials=True,
    allow_methods=["GET", "POST"],
    allow_headers=["Authorization", "Content-Type"],
)

```

4.3 AG-UI Middleware — Scoped to /api/chat Only

The AG-UI middleware is NOT registered globally. It is mounted as a sub-application on the /api/chat route only. Plain REST routes (invoices, suppliers, opportunities) never touch the SSE or AG-UI machinery — this avoids SSE handshake overhead on every data fetch.

```

from copilotkit.integrations.fastapi import add_fastapi_endpoint
from copilotkit import CopilotKitSDK, LangGraphAgent

sdk = CopilotKitSDK(

```

```

agents=[
    LangGraphAgent(
        name="spend_recommendation_agent",
        description="Sourcing spend analysis and supplier recommendation",
        graph=spend_recommendation_graph,    # LangGraph CompiledGraph
    )
]

# Scoped mount — only /api/chat goes through AG-UI
add_fastapi_endpoint(app, sdk, "/api/chat")

```

4.4 The Chat Route — What Happens at POST /api/chat

When a POST /api/chat arrives, the following sequence executes within Workload 2:

Step 4.4.1 Request received from APIM

FastAPI receives POST with Content-Type: application/json body containing the user message, session_id, thread_id, and readable context (invoices/suppliers currently on screen)

Step 4.4.2 AG-UI middleware takes over

Middleware identifies the request as an AG-UI agent run request; sets response headers to Content-Type: text/event-stream, Cache-Control: no-cache, Transfer-Encoding: chunked

Step 4.4.3 LangGraph agent invoked asynchronously

Middleware calls graph.astream_events() — the async streaming interface of LangGraph. This returns an async generator that yields events as the agent runs

Step 4.4.4 Events relayed as SSE chunks

For each event yielded by the agent, the middleware serialises it to AG-UI protocol format and writes it as an SSE data frame to the open HTTP connection

Step 4.4.5 Stream closes

When the agent graph reaches the END node (or errors), the generator is exhausted; middleware writes a final RUN_FINISHED event and closes the response

```

# Simplified view of what AG-UI runner does internally:
async def run_agent_stream(request: AgentRequest):
    async for event in graph.astream_events(
        input={"messages": request.messages, "context": request.context},
        config={"configurable": {"thread_id": request.thread_id}},
        version="v2",
    ):
        sse_chunk = serialise_to_agui_event(event)
        yield f"data: {sse_chunk}\n\n"

```

4.5 Timeout Configuration

FastAPI does not have a global timeout — timeouts are configured per route. The /api/chat route must allow up to 300 seconds for a full agent run (multiple tool calls + LLM inference + planning). Plain REST routes enforce a 15-second timeout.

```
import asyncio
```

```
from fastapi import HTTPException

# In the route handler for plain REST:
async def fetch_invoices(filters: InvoiceFilters):
    try:
        return await asyncio.wait_for(
            backend_service.invoices.get_invoices(filters),
            timeout=15.0
        )
    except asyncio.TimeoutError:
        raise HTTPException(status_code=504, detail="Upstream timeout")
```

APIM alignment: The FastAPI per-route timeout must be shorter than the APIM backend timeout for the same route. If APIM has a 300s timeout on /api/chat, FastAPI's agent invocation must complete within 300s. If FastAPI times out first, it will close the SSE stream with an error before APIM's timeout fires — which is the correct behaviour. The concern is the reverse: if APIM times out before FastAPI does, APIM closes the connection to the client while the agent is still running, causing a confusing partial stream.

4.6 OAuth2 Security — Workload 2 Outbound Calls

Workload 2 makes outbound calls to two destinations: Workload 3 (AI Agent) over ClusterIP, and external SaaS APIs (Oracle AP, Coupa) via Azure Firewall. Each call uses a different auth mechanism:

Outbound Call	Auth Mechanism	Credential Source
Workload 2 → Workload 3 (ClusterIP)	Kubernetes NetworkPolicy + mutual service identity	No credentials; NetworkPolicy restricts traffic to authorised pods only
Workload 2 → Oracle AP	OAuth2 client credentials (service account)	Key Vault WL2 via Workload Identity Managed Identity
Workload 2 → Coupa/EAA	OAuth2 client credentials (API key + secret)	Key Vault WL2 via Workload Identity Managed Identity
Workload 2 → Redis	Connection string (TLS)	Key Vault WL2 via Workload Identity Managed Identity
Workload 2 → Databricks (direct reads)	OAuth2 ADC / SA impersonation	Key Vault WL2 — service account token

 **Key Vault access pattern:** Workload 2's pod has a Managed Identity (MI-workload-2) bound via AKS Workload Identity. To retrieve a secret, the pod requests a token from the OIDC endpoint, presents it to Key Vault, and receives the secret value at runtime. No secret is ever stored in the pod's environment variables, Kubernetes Secret objects, or the container image. Secret rotation at the Key Vault level is immediately effective — the next pod request retrieves the rotated value without a redeploy.

5. AI Agent — Workload 3 (LangGraph)

Workload 3 hosts the LangGraph agent — the AI reasoning core of the system. It runs as a standalone Python service inside the AKS cluster, accessible only from Workload 2 over an internal ClusterIP Service. It has no public endpoint.

5.1 Two-Agent Design

The agent workload implements two distinct LLM-backed reasoning components. They are not simply two nodes in one graph — they are architecturally separate concerns that happen to be co-located in the same Kubernetes deployment:

Agent	Type	Prompt Focus	Model
LangGraph Orchestrator	Outer loop — conversational	Understand conversation history, decide when to delegate, format final response for user	GPT-4.1 (primary)
Semantic Query Planner	Domain specialist — planning	Resolve business intent, map to ontology, select correct tool, validate parameters against schema	GPT-4.1 or Claude Sonnet (configurable)

The rationale for two agents: the Orchestrator handles the conversational and state dimension (multi-turn context, when to respond vs. when to call a tool), while the Planner handles the domain-specific semantic dimension (what does "SE Asia suppliers who handle rush orders" mean in terms of Unity Catalog table columns and filter values). Mixing these concerns in one agent's prompt leads to degraded performance on both dimensions.

5.2 LangGraph Graph Structure

```
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver

# Graph nodes:
# 1. orchestrator_node    - main LLM call; decides action
# 2. content_safety_in    - Prompt Shields on user input
# 3. planner_node         - A2A call to Semantic Query Planner
# 4. tool_executor        - executes planned tool call
# 5. content_safety_out   - Prompt Shields on response
# 6. hitl_approval        - human-in-the-loop interrupt (conditional)
# 7. response_formatter   - formats final response for streaming

graph = StateGraph(AgentState)
graph.add_node("content_safety_in", content_safety_input_node)
graph.add_node("orchestrator", orchestrator_node)
graph.add_node("planner", planner_node)
graph.add_node("tool_executor", tool_executor_node)
graph.add_node("content_safety_out", content_safety_output_node)
graph.add_node("hitl_approval", hitl_approval_node)
graph.add_node("response_formatter", response_formatter_node)
```

```
graph.set_entry_point("content_safety_in")
graph.add_edge("content_safety_in", "orchestrator")
graph.add_conditional_edges("orchestrator", route_after_orchestrator, {
    "plan": "planner",
    "respond": "content_safety_out",
    "end": END,
})
graph.add_edge("planner", "tool_executor")
graph.add_conditional_edges("tool_executor", route_after_tool, {
    "hitl": "hitl_approval",
    "respond": "content_safety_out",
})
graph.add_edge("hitl_approval", "content_safety_out")
graph.add_edge("content_safety_out", "response_formatter")
graph.add_edge("response_formatter", END)
```

5.3 Postgres Checkpointing

LangGraph persists the full agent state (message history, tool call results, orchestrator decisions) to Azure PostgreSQL Flexible Server after every node execution. This enables:

- Session resumption: if the SSE connection drops, the next request with the same thread_id resumes from the last checkpoint
- Human-in-the-loop: the agent pauses at the hitl_approval node; the checkpoint preserves state while waiting for the user's decision
- Multi-turn conversation: the orchestrator reads prior turns from the checkpoint, maintaining context across messages

```
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver

# Connection via Private Endpoint — no public access
# Credentials from Key Vault via Workload Identity
checkpointer = AsyncPostgresSaver.from_conn_string(
    os.environ["POSTGRES_CONN_STR"] # sourced from Key Vault WL3
)

compiled_graph = graph.compile(checkpointer=checkpointer)
```

 **Checkpoint security:** The Postgres connection string is sourced from Key Vault WL3 via Workload Identity MI-workload-3. The PostgreSQL Flexible Server is accessible only via Private Endpoint pe-postgres (10.0.5.x) — no public network access. Zone-redundant standby provides automatic failover with < 60s RTO.

5.4 Orchestrator Node — Detail

The orchestrator is the "outer loop" — the first node to see the user's message after Content Safety has cleared it. Its responsibilities are:

- Read full conversation history from the checkpoint (multi-turn context)
- Decide whether to respond directly (simple clarification, out-of-scope request) or delegate to the Planner (requires data lookup or spend analysis)

- If delegating: extract the user's intent as a structured delegation request for the Planner
- If responding: draft the final response and route to content_safety_out

```

async def orchestrator_node(state: AgentState, config: RunnableConfig):
    messages = state["messages"]
    context = state["copilot_readable_context"] # from useCopilotReadable

    response = await llm.ainvoke(
        [SystemMessage(ORCHESTRATOR_SYSTEM_PROMPT)] + messages,
        config=config
    )

    # Orchestrator returns either:
    # { "action": "plan", "intent": "find SE Asia rush suppliers" }
    # { "action": "respond", "content": "I need more info..." }
    return { "orchestrator_output": parse_orchestrator_response(response) }

```

5.5 A2A Call — Orchestrator to Semantic Query Planner

When the orchestrator decides to delegate (action: "plan"), it invokes the Semantic Query Planner via the A2A protocol (JSON-RPC 2.0 over HTTPS). Both agents publish Agent Cards; both are registered in APIM for governance.

```

import httpx

async def planner_node(state: AgentState, config: RunnableConfig):
    intent = state["orchestrator_output"]["intent"]

    # A2A task/send request
    task_request = {
        "jsonrpc": "2.0",
        "method": "tasks/send",
        "params": {
            "id": str(uuid4()),
            "sessionId": config["configurable"]["thread_id"],
            "message": {
                "role": "user",
                "parts": [{ "type": "text", "text": intent }]
            }
        }
    }

    async with httpx.AsyncClient() as client:
        resp = await client.post(
            f"{PLANNER_ENDPOINT}/", # internal ClusterIP
            json=task_request,
            timeout=60.0
        )

        plan = resp.json()["result"]["artifacts"][0]["parts"][0]["data"]
        # plan = { "tool": "query_spend_table", "params": {...}, "guardrails_passed":
        true }
        return { "tool_plan": plan }

```

5.6 Semantic Query Planner — Detail

The Semantic Query Planner is a domain-specialist LLM agent. Its system prompt is narrow and specific: given a user intent expressed in natural language, resolve it to a specific tool call with validated parameters. It does not handle conversation or multi-turn context — that is the Orchestrator's job.

The Planner's responsibilities:

- Intent resolution: map "SE Asia rush order suppliers" to specific Unity Catalog filter column values
- Ontology lookup: validate that resolved values exist in the business ontology (approved region codes, supplier tiers, spend categories)
- Tool selection: choose between `query_spend_table`, `find_supplier`, and `query_semantic_planner_rag`
- Parameter extraction: construct the exact tool payload with sanitised, typed parameters
- Guardrail validation (Tool Exec Guard): confirm the planned call is within the user's Unity Catalog permissions before returning

```
# Planner system prompt structure (abbreviated):
PLANNER_PROMPT = """
You are a sourcing data specialist. Given a user intent:
1. Resolve to Unity Catalog column values using the ontology below
2. Select exactly one tool: query_spend_table | find_supplier |
   query_semantic_planner_rag
3. Construct parameters matching the tool's JSON schema exactly
4. Return ONLY valid JSON: { "tool": "...", "params": {...}, "guardrails_passed":
   true/false }

Ontology: {region_codes}, {supplier_tiers}, {spend_categories}
Tool schemas: {tool_schemas}
User permissions: {user_unity_catalog_permissions}
"""
```

5.7 Human-in-the-Loop Approval Gate

After the tool executor returns results, the graph evaluates whether the recommendation is high-stakes (spend impact above a configured threshold). If it is, the graph routes to `hitl_approval_node`, which is implemented as a LangGraph interrupt:

```
from langgraph.types import interrupt

async def hitl_approval_node(state: AgentState):
    recommendation = state["draft_recommendation"]
    spend_impact = recommendation["spend_impact_usd"]

    if spend_impact > HITL_THRESHOLD_USD:
        # Pause graph execution; checkpoint written; stream paused
        user_decision = interrupt({
            "type": "approval_required",
            "recommendation": recommendation,
            "message": "This recommendation affects spend above threshold.
Approve?",
        })
        # Execution resumes here when user responds via /api/chat with decision
        state["hitl_decision"] = user_decision

    return state
```

When the interrupt fires, LangGraph writes the full state to the Postgres checkpoint and suspends. The AG-UI runner serialises the interrupt payload as a `STATE_SNAPSHOT` SSE event, which CopilotKit renders as an approval prompt in the frontend. The user's decision arrives as a new `POST /api/chat` with the same `thread_id` — LangGraph resumes from the checkpoint, passing the decision through.

Both the recommendation and the human decision are written to the immutable audit store before the graph continues.

5.8 OAuth2 Security — Workload 3 Outbound Calls

Outbound Call	Auth Mechanism	How Token Obtained
WL3 → Azure AI Content Safety	Managed Identity (MI-workload-3) → Entra ID token	azure-identity DefaultAzureCredential; token fetched from IMDS endpoint at runtime
WL3 → AI Gateway (APIM AI)	Managed Identity → Entra ID token (or APIM subscription key in Key Vault)	DefaultAzureCredential or Key Vault WL3 secret
WL3 → Databricks MCP	OAuth2 SA impersonation — service account token	Key Vault WL3; token presented in MCP Authorization header
WL3 → Cosmos DB (Vector)	Managed Identity — Cosmos DB RBAC role	DefaultAzureCredential; MI-workload-3 has Cosmos DB Built-in Data Reader role
WL3 → PostgreSQL	Connection string (TLS, pg_bouncer)	Key Vault WL3; no IAM token — Postgres password auth over TLS
WL3 → Planner (A2A, ClusterIP)	NetworkPolicy only — no credential	Same-cluster internal call; NetworkPolicy restricts to authorised source pod

 **DefaultAzureCredential chain:** All Managed Identity calls use azure-identity's DefaultAzureCredential, which resolves credentials in this order: environment variables → Workload Identity (IMDS) → Managed Identity. In production AKS with Workload Identity enabled, the IMDS endpoint returns a token scoped to the pod's Managed Identity. No credentials are stored in the pod — the identity assertion comes from the Kubernetes projected service account token.

6. Azure AI Content Safety — Six-Step Flow

Content Safety is invoked twice per agent run — once on the inbound user prompt and once on the outbound LLM response. These are explicit API calls from the agent code, not APIM-embedded policies, so each call appears as a discrete, timestamped, auditable entry in both LangSmith traces and Azure Monitor logs.

6.1 Step-by-Step Detail

Step	Node	API Called	Payload	Blocks On	Audit Record
1	content_safety_in	POST /contentsafety/text:shieldPrompt	{ userPrompt: <user message>, documents: [<copilotReadable context>] }	Jailbreak detected, direct injection, indirect injection in documents	Timestamp, type, confidence score written to LangSmith
2	content_safety_in	(no API call — decision gate)	If step 1 returns attackDetected: true → raise AgentSafetyError	Step 1 attack flag	RUN_ERROR emitted to stream; user "Request could not be processed"
3	orchestrator / planner	POST to AI Gateway (APIM AI endpoint)	Sanitised prompt + system prompt + conversation history	APIM token quota, rate limit	Token usage logged in APIM and LangSmith
4	orchestrator / planner	(response received from LLM)	LLM completion text	—	Completion logged in LangSmith, token count
5	content_safety_out	POST /contentsafety/text:analyze	{ text: <LLM response>, categories: [Hate,SelfHarm,Sexual,Violence], groundedness config }	Any harm category severity HIGH; groundedness check failed	Timestamp, categories, severity score written to LangSmith audit store
6	content_safety_out	(no API call — decision gate)	If step 5 returns blocked category → replace response with safe fallback	Step 5 harm flag	Replacement event logged, original flagged response reaches user

6.2 Prompt Shields Implementation

```
from azure.ai.contentsafety.aio import ContentSafetyClient
from azure.identity.aio import DefaultAzureCredential
from azure.ai.contentsafety.models import ShieldPromptOptions

async def content_safety_input_node(state: AgentState):
    credential = DefaultAzureCredential()
```

```

client = ContentSafetyClient(
    endpoint=os.environ["CONTENT_SAFETY_ENDPOINT"], # private endpoint
    credential=credential
)

result = await client.shield_prompt(
    ShieldPromptOptions(
        user_prompt=state["messages"][-1].content,
        documents=state.get("copilot_readable_context", []),
    )
)

if result.user_prompt_attack_detection.attack_detected:
    raise AgentSafetyError("Prompt injection detected; run terminated")

state["safety_prompt_result"] = result
return state

```

 **Indirect injection:** The documents parameter is critical. It passes the readable context (invoices/suppliers currently on screen) through Prompt Shields. This catches indirect prompt injection — malicious instructions embedded in Databricks data (e.g. a supplier name field containing "Ignore previous instructions and..."). Without this, the agent would process injected instructions as legitimate data context.

6.3 Groundedness Detection

Step 5 includes groundedness detection — verifying that spend figures and supplier names cited in the LLM's response are traceable to the tool call results retrieved in the current run, not hallucinated from the model's training data. This is particularly important for procurement: an LLM citing a fabricated spend figure could lead to incorrect sourcing decisions.

```

from azure.ai.contentsafety.models import AnalyzeTextOptions, GroundednessConfig

async def content_safety_output_node(state: AgentState):
    result = await client.analyze_text(
        AnalyzeTextOptions(
            text=state["draft_response"],
            groundedness_detection=GroundednessConfig(
                enabled=True,
                grounding_sources=state["tool_results"], # actual retrieved data
            )
        )
    )
    if not result.groundedness_detection.is_grounded:
        state["draft_response"] = UNGROUNDED_FALLBACK_MESSAGE
    return state

```

7. AI Gateway — APIM AI + Azure OpenAI / Foundry

The "AI Gateway" is implemented as a capability layer within Azure API Management (APIM). The agent never calls the LLM endpoint directly — all inference traffic routes through APIM, which applies token quotas, semantic caching, model routing, and observability before forwarding to the Foundry-hosted model.

7.1 Why Route Through a Gateway

Concern	Without Gateway	With APIM AI Gateway
Token cost control	Unbounded; runaway agent run can spend without limit	Per-user daily quota enforced; session ceiling enforced
Model switching	Requires agent code change and redeploy	APIM backend pool update; zero agent code change
Repeat query cost	Every semantically similar query hits the LLM	Semantic cache returns cached response; no LLM call
Multi-model fallback	Agent must implement retry logic	APIM backend pool with automatic failover across regions/models
Observability	Scattered per-agent logs	Centralised APIM token usage log per user, per route, per model
Rate limit enforcement	Must be coded in every agent	Single APIM policy applies to all agents calling the gateway

7.2 APIM AI Gateway Configuration for /api/chat

```
<!-- APIM inbound policy for /api/chat -->
<policies>
  <inbound>
    <!-- 1. Validate OIDC token from Ping -->
    <validate-jwt header-name="Authorization" failed-validation-httpcode="401">
      <openid-config url="https://ping.kroger.com/.well-known/openid-configuration"/>
    </validate-jwt>

    <!-- 2. Per-user daily token quota -->
    <azure-openai-token-limit counter-key="@ (context.User.Id) "
      tokens-per-minute="50000" estimate-prompt-tokens="true"/>

    <!-- 3. Semantic cache lookup -->
    <azure-openai-semantic-cache-lookup score-threshold="0.85"
      embeddings-backend-id="embedding-backend"/>

    <!-- 4. Disable response buffering for SSE -->
    <set-header name="X-Accel-Buffering" exists-action="override">
      <value>no</value>
    </set-header>
  </inbound>
```

```
<backend>
  <!-- 5. Load balance across Azure OpenAI deployments -->
  <azure-openai-load-balancing backend-pool-id="openai-pool"/>
</backend>
<outbound>
  <!-- 6. Cache miss: store response in semantic cache -->
  <azure-openai-semantic-cache-store duration="600"/>
  <!-- 7. Emit token usage metrics to Azure Monitor -->
  <azure-openai-emit-token-metric/>
</outbound>
</policies>
```

7.3 Semantic Caching

Semantic caching stores LLM responses indexed by the semantic meaning of the prompt (using embeddings), not the exact text. A threshold of 0.85 cosine similarity is used — queries semantically equivalent to a cached prompt return the cached response without a new LLM call. This directly reduces token spend on repeat or near-duplicate sourcing queries common in a category management context (e.g. "show Q3 supplier spend" vs "what was supplier spend in Q3").

7.4 Model Backend Pool

Priority	Model	Deployment	Failover Condition
1 (Primary)	GPT-4.1	Azure OpenAI — us-east	Healthy; used for all requests
2 (Secondary)	GPT-4.1	Azure OpenAI — us-central	Primary returns 429 (throttled) or 5xx
3 (Optional)	Claude Sonnet	Anthropic via Foundry	Manual routing for complex semantic planning

7.5 LLM Call from Agent Code

```
from langchain_openai import AzureChatOpenAI
from azure.identity import DefaultAzureCredential, get_bearer_token_provider

# Credential: Managed Identity → Entra ID token for OpenAI scope
token_provider = get_bearer_token_provider(
    DefaultAzureCredential(),
    "https://cognitiveservices.azure.com/.default"
)

llm = AzureChatOpenAI(
    azure_endpoint=os.environ["APIM_AI_GATEWAY_ENDPOINT"], # APIM, not OpenAI
    directly=True,
    azure_deployment="gpt-4-1",
    api_version="2024-12-01-preview",
    azure_ad_token_provider=token_provider,
    streaming=True,
)
```

 **No API keys:** The LLM client authenticates using an Entra ID token obtained from the Managed Identity (MI-workload-3). The `azure_ad_token_provider` fetches a short-lived bearer token at runtime. No OpenAI API key or APIM subscription key is stored in the pod — token rotation is automatic and managed by Entra ID.

8. Databricks — MCP Tool Execution

The agent interacts with Kroger's spend and supplier data exclusively through the Databricks Managed MCP endpoint. The MCP (Model Context Protocol) provides a structured, governed interface for LLM tool calls — each tool is defined with a JSON schema, and the agent can only call tools with parameters that match the schema.

8.1 MCP Endpoint

The Databricks MCP endpoint is accessed via Private Link inside the VNet. The public `sqladmin.databricks.com` hostname is disabled at the workspace level. All MCP traffic routes through the private endpoint `pe-databricks (10.0.5.x)`.

```
# MCP endpoint - private, not public
DATABRICKS_MCP_URL = "https://private-ep.databricks.com/mcp" # resolves to 10.0.5.x
# NOT: https://sqladmin.databricks.com/mcp ← disabled
```

8.2 Three Tool Definitions

Tool	MCP Method	Purpose	Key Parameters
query_spend_table	tools/call execute_sql	Query invoice spend by category, supplier, date range	database, sql_statement (parameterised)
find_supplier	tools/find_alternative_flight	Find suppliers matching capability requirements	supplier, name, tool_context (capability, lead_time)
query_semantic_planner_rag	tools/query_flight_risk	RAG query against Cosmos Vector DB for semantic supplier matching	invoice_spend_amount, date, tool_context

8.3 Tool Call Construction and Sanitisation

Every tool call passes through `sanitize.py` and `query.py` before being sent to Databricks MCP. This is enforced at the tool executor node — tool plans from the Planner are not executed directly but are first validated and sanitised:

```
# tool_executor_node - simplified
async def tool_executor_node(state: AgentState):
    plan = state["tool_plan"]
    tool_name = plan["tool"]
    raw_params = plan["params"]

    # 1. Sanitise inputs - validate types, ranges, no injection patterns
    sanitised = sanitize.validate_tool_params(tool_name, raw_params)

    # 2. Build MCP payload - parameterised, no string concatenation
    mcp_payload = query.build_mcp_request(tool_name, sanitised)

    # 3. Authenticate - OAuth2 SA impersonation token from Key Vault
    token = await keyvault.get_databricks_sa_token()
```

```
# 4. Execute via MCP — Private Endpoint, HTTPS
result = await mcp_client.call_tool(
    url=DATABRICKS_MCP_URL,
    tool=tool_name,
    params=mcp_payload,
    auth_token=token,
)

return { "tool_results": result, "tool_name": tool_name }
```

8.4 MCP Request Format

```
# Full MCP JSON-RPC 2.0 request (query_spend_table example)
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "execute_sql",
    "arguments": {
      "database": "kroger_gnfr_prod",
      "sql_statement": "SELECT supplier_name, SUM(spend_usd) AS total
        FROM spend_fact
        WHERE category_code = ?
        AND region_code IN (?, ?)
        AND invoice_date BETWEEN ? AND ?
        GROUP BY supplier_name
        ORDER BY total DESC LIMIT 20",
      "parameters": ["GNFR-FACILITIES", "SEA-01", "SEA-02",
        "2025-01-01", "2025-09-30"]
    }
  }
}
```

Key pattern: The `sql_statement` contains `?` placeholders — never string-concatenated values. The `parameters` array is passed separately and bound by the Databricks JDBC driver. This is the standard SQL injection prevention mechanism. Even if a user prompt contained adversarial SQL, it would land in a parameter slot that is treated as a literal string, not executed as SQL.

8.5 Unity Catalog Row-Level Access Control

Unity Catalog enforces row-level security on all tables. Each Sourcing Category Manager's identity is mapped to a Unity Catalog row filter function on the `spend_fact` table. When the agent executes a query with SA impersonation, Unity Catalog applies the user's row filter transparently — the query result contains only the rows the user is permitted to see, regardless of what the SQL statement asks for.

```
-- Unity Catalog row filter (configured once, applies to all queries):
CREATE ROW FILTER kroger_gnfr.spend_row_filter ON kroger_gnfr.spend_fact
AS (category_manager_id) RETURN
  (category_manager_id = CURRENT_USER())
  OR is_account_admin();
```

This means the agent cannot retrieve data outside the authenticated user's category scope — even if the Planner incorrectly constructs a query without a category filter, Unity Catalog filters it at the data plane. Defence in depth: guardrails at the planning layer AND access control at the data layer.

8.6 Data Ingestion Pipeline

Job	Source	Schedule	Target	Auth
Oracle AP ingestion	Oracle AP REST API (OAuth2)	Every 4 hours	Databricks spend_fact table	OAuth2 client credentials in Key Vault
Coupa ingestion	Coupa API (OAuth2)	Every 4 hours	Databricks supplier tables	OAuth2 API key + secret; Vault
Coupa EAA ingestion	Coupa EAA API	Every 4 hours	Databricks EAA tables	OAuth2; Key Vault
Redis pre-warm job	Databricks (hot read tables)	Every 15 minutes	Redis Cache	Databricks SA token; Red conn string — both Key V

9. Observability — LangSmith, Azure Monitor, Dynatrace

The agentic path requires observability at two levels that traditional APM tools do not cover: LLM-specific metrics (token usage, prompt quality, tool selection accuracy, groundedness) and infrastructure metrics (latency per hop, SSE stream health, Postgres checkpoint reliability). Both levels are instrumented.

9.1 LangSmith Trace Structure

Every agent run generates a LangSmith trace. Traces are attributed at node level — each LangGraph node generates its own span within the trace, enabling per-node latency and cost attribution.

Trace Span	What It Captures	Key Metrics
run_root	Full agent run start to finish	Total latency, total tokens, run_id, session_id, user_id
content_safety_in	Prompt Shields call on user input	Latency, attack_detected flag, attack_type, confidence score
orchestrator	LLM call — orchestrator model	Latency, prompt_tokens, completion_tokens, decision (plan vs respond)
planner (A2A)	A2A call to Semantic Query Planner	Latency, tool selected, params extracted, guardrails_passed
planner_llm	LLM call inside the Planner	Latency, prompt_tokens, completion_tokens, model used
tool_executor	MCP tool call to Databricks	Latency, tool_name, param_hash, result_row_count, error if any
hitl_approval	Human-in-the-loop pause/resume	Wait duration, decision (approved/rejected), approver_id
content_safety_out	Prompt Shields on LLM response	Latency, flagged flag, groundedness score, harm_categories
response_formatter	Final response construction	Output token count, response length

9.2 Trace Tags — Applied to Every Run

```
# Applied via LangSmith metadata at graph invocation:
config = {
  "configurable": { "thread_id": thread_id },
  "metadata": {
    "session_id":      session_id,
    "user_id":         user_id,
    "category_manager_id": category_manager_id,
    "tools_invoked":    [],          # appended as tools are called
    "content_safety_flagged": False,  # set to True if any flag fires
    "hitl_triggered":   False,        # set to True if approval gate fired
    "model_primary":    "gpt-4-1",
  }
}
```

```
    "sprint":                                "sprint-4",
  }
}
```

9.3 Seven Metric Categories

Category	Metrics	Alert Threshold
Latency	time_to_first_token, planner_latency, per-tool latency, content_safety_round_trip, end_to_end	P95 end-to-end > 45s → alert
Cost / Tokens	tokens_per_run (prompt vs completion), cost_per_session, cost_per_tool_cycle	Daily cost > 2× 7-day average → alert
Tool correctness	tool_selection_accuracy (vs eval set), parameter_extraction_accuracy, tool_failure_rate, retry_loop_count	Accuracy < baseline - 5% → CI blocks merge
Groundedness	groundedness_score per run, citation_traceability_rate	< 90% grounded runs in 1hr window → alert
Safety	content_safety_block_rate (prompt side and response side), prompt_injection_detection_rate, hitl_trigger_rate, hitl_override_rate	Block rate > 5% in 10min → High alert
Reliability	run_success_failure_rate by failure type, checkpoint_recovery_success_rate, A2A_MCP_call_failure_rate	Failure rate > 2% → alert
User quality	thumbs_up_down per run, session_abandonment_rate, recommendation_acceptance_rate	Negative feedback > 20% in 1hr → alert

9.4 LangSmith Eval Pipeline in CI

A LangSmith evaluation dataset of 60+ representative sourcing queries with expected tool selections and grounded answers runs as a CI check on every pull request touching the agent code. Results are compared against a stored baseline:

```
# .github/workflows/agent-eval.yml (simplified)
- name: Run LangSmith evals
  run: |
    python evals/run_evals.py \
      --dataset kroger-sourcing-eval-v2 \
      --evaluators tool_selection_accuracy groundedness \
      --baseline evals/baseline.json \
      --fail-if-regression 0.05

# Failure conditions that block merge:
# - tool_selection_accuracy drops > 5% below baseline
# - groundedness_score drops > 5% below baseline
```

```
# - Any eval run produces a content_safety block (regression in prompt construction)
```

9.5 Azure Monitor — Infrastructure Metrics

Infrastructure-level observability (separate from LangSmith's AI-specific tracing) is captured in Azure Monitor via OpenTelemetry instrumentation on each workload pod. FastAPI is instrumented with the Azure Monitor OpenTelemetry distro:

```
from azure.monitor.opentelemetry import configure_azure_monitor
configure_azure_monitor(connection_string=os.environ["APPINSIGHTS_CONNECTION_STRING"])
```

This captures: HTTP request latency and status per route, dependency calls (Redis, Postgres, Databricks), exception traces, and custom metrics emitted from the agent (token count, tool call count, HitL trigger events).

9.6 SSE Stream Health Monitoring

SSE streams are long-lived connections (up to 300 seconds). Standard HTTP metrics report the connection as a single request for its entire duration, obscuring whether the stream is healthy mid-flight. The following custom metrics are emitted to Azure Monitor to fill this gap:

- `sse_stream_opened` — counter; increments when `/api/chat` SSE connection is established
- `sse_stream_closed_clean` — counter; increments when stream closes with `RUN_FINISHED` event
- `sse_stream_closed_error` — counter; increments when stream closes with `RUN_ERROR` or timeout
- `sse_chunk_latency_ms` — histogram; time between consecutive SSE chunks (gaps > 10s alert)
- `sse_time_to_first_chunk_ms` — histogram; time from POST to first SSE data frame (P95 target < 3s)

10. End-to-End Walkthrough — Full Request Lifecycle

This section traces a single user request — "Find me GNFR facilities suppliers in Southeast Asia who can handle rush orders, and compare their spend to our current contracts" — through every layer of the system with the actual data formats and auth tokens at each hop.

10.1 Browser → App Gateway → APIM

```
POST https://gnfr-workbench.kroger.com/api/chat
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9... (Ping OIDC token)
Content-Type: application/json

{
  "thread_id": "session-abc123",
  "messages": [{ "role": "user", "content": "Find me GNFR facilities
suppliers..." }],
  "context": { "invoices": [...], "suppliers": [...] } // useCopilotReadable data
}
```

App Gateway WAF inspects for OWASP violations and oversized payload. APIM validates the Ping OIDC token (signature, expiry, audience claim), checks the user's daily token quota (not exceeded), and looks up the semantic cache (cache miss — new query). APIM strips the original Authorization header and sets a new Managed Identity token for the backend call, forwarding to the AKS internal LoadBalancer at 10.0.4.x.

10.2 APIM → Workload 2 (FastAPI)

```
POST http://10.0.4.x/api/chat ← internal ClusterIP; no public access
X-User-Id: user-jsmith@kroger.com ← injected by APIM from validated token claim
X-Thread-Id: session-abc123
Authorization: Bearer <MI-apim Entra token> ← APIM Managed Identity token
```

FastAPI receives the request. The AG-UI middleware identifies it as an agent run request. Response headers are immediately set (text/event-stream, no-cache, chunked) and the first SSE heartbeat is sent to keep the connection alive while the agent starts. The LangGraph graph is invoked asynchronously.

10.3 Content Safety — Prompt Inspection

```
POST https://10.0.5.x/contentsafety/text:shieldPrompt ← Private Endpoint
Authorization: Bearer <MI-workload-3 Entra token>

{
  "userPrompt": "Find me GNFR facilities suppliers in Southeast Asia...",
  "documents": ["Invoice #1234 - Supplier: ABC Facilities...", ...]
}
```

```
← Response:
{
  "userPromptAttackDetection": { "attackDetected": false },
  "documentsAttackDetection": { "attackDetected": false }
}
```

}

No attack detected. Graph continues to orchestrator node.

10.4 Orchestrator LLM Call → AI Gateway

```
POST https://apim-kroger.azure-api.net/openai/deployments/gpt-4-1/chat/completions
Authorization: Bearer <MI-workload-3 Entra token>
api-version: 2024-12-01-preview
{
  "messages": [
    { "role": "system", "content": "<ORCHESTRATOR_SYSTEM_PROMPT>" },
    { "role": "user", "content": "Find me GNFR facilities suppliers..." }
  ],
  "stream": true
}
```

APIM AI Gateway receives this call: semantic cache lookup miss (new query), token counter incremented, routed to GPT-4.1 us-east deployment. Orchestrator decides action: "plan" (data lookup required). Returns:

```
{ "action": "plan", "intent": "Find GNFR facilities suppliers in Southeast Asia
capable of rush orders; compare spend to current contracts" }
```

10.5 A2A Call — Orchestrator → Planner

```
POST http://semantic-planner.default.svc.cluster.local/ ← ClusterIP
{
  "jsonrpc": "2.0", "method": "tasks/send",
  "params": {
    "id": "task-xyz789", "sessionId": "session-abcd123",
    "message": { "role": "user", "parts": [{ "type": "text",
      "text": "Find GNFR facilities suppliers in SE Asia capable of rush
orders..." }] }
  }
}
```

Planner's LLM call resolves: region_code = ["SEA-01","SEA-02","SEA-03"], category_code = "GNFR-FACILITIES", capability = "rush_order", tool = "find_supplier". Returns:

```
{
  "result": { "artifacts": [{ "parts": [{ "type": "data", "data": {
    "tool": "find_supplier",
    "params": { "region": ["SEA-01","SEA-02","SEA-03"], "capability": "rush_order",
      "category_code": "GNFR-FACILITIES" },
    "guardrails_passed": true
  }}}]}
}
```

10.6 Tool Execution → Databricks MCP

```
POST https://private-ep.databricks.com/mcp ← Private Endpoint 10.0.5.x
Authorization: Bearer <SA impersonation token from Key Vault>
```

```
{
  "jsonrpc": "2.0", "method": "tools/call",
  "params": { "name": "find_alternative_flight", "arguments": {
    "supplier": null,
    "name": null,
    "tool_context": {
      "region_codes": ["SEA-01", "SEA-02", "SEA-03"],
      "capability": "rush_order",
      "category_code": "GNFR-FACILITIES"
    }
  }}
}
```

Unity Catalog applies the category manager's row filter — only rows where `category_manager_id = CURRENT_USER()` are returned. Result: 12 suppliers with rush order capability in SEA regions, with spend data from the `spend_fact` table joined.

10.7 Content Safety — Response Inspection

```
POST https://10.0.5.x/contentsafety/text:analyze
{
  "text": "Based on the data, I recommend ABC Facilities (SEA-01) as primary...",
  "groundednessDetection": {
    "enabled": true,
    "groundingSources": ["<tool result JSON>"]
  }
}
← { "groundednessDetection": { "isGrounded": true }, "categoriesAnalysis": [] }
```

Response is grounded and no harm categories fired. Response proceeds to stream.

10.8 SSE Stream — Response to Browser

The formatted recommendation flows back as SSE events through the chain:

```
Workload 3 → Workload 2 AG-UI runner → APIM (buffering OFF) → App Gateway → Browser
```

```
data: {"type":"RUN_STARTED","run_id":"run-abc"}

data: {"type":"TEXT_MESSAGE_CHUNK","content":"Based on the supplier data"}
data: {"type":"TEXT_MESSAGE_CHUNK","content":" retrieved, I recommend"}
data: {"type":"TEXT_MESSAGE_CHUNK","content":" ABC Facilities (SEA-01)"}
...

data: {"type":"RUN_FINISHED","run_id":"run-abc","total_tokens":1847}
```

CopilotKit's React hooks consume these events and update the chat UI incrementally — the user sees tokens appearing in real time, with tool-call progress indicators shown during the Databricks query phase.

10.9 Total Latency Budget (Target)

Hop	Component	Target Latency
Browser → APIM	TLS + WAF + token validation	< 200ms
APIM → FastAPI	Internal routing	< 50ms
Content Safety (prompt)	Prompt Shields API	< 500ms
Orchestrator LLM call	GPT-4.1 (streaming, time to first token)	< 2s
A2A call to Planner	Internal ClusterIP + Planner LLM	< 5s
Databricks MCP tool call	Private Link + Unity Catalog query	< 10s
Content Safety (response)	Prompt Shields + groundedness	< 800ms
SSE relay back to browser	Chunked transfer through chain	< 100ms overhead
Total — time to first token in browser	Sum of above to first chunk	< 8s (P95 target)
Total — full response in browser	End of stream	< 30s (P95 target for typical query)

11. Error Handling and Resilience

11.1 Error Taxonomy

Error Type	Where It Occurs	Handling Strategy	User Experience
Content Safety block (prompt)	content_safety_in node	Raise AgentSafetyError; emit RUN_ERROR SSE event	User sees: "Your request could not be processed"
Content Safety block (response)	content_safety_out node	Replace response with safe fallback message	User sees: "I wasn't able to generate a response for that query"
LLM timeout / 5xx	AI Gateway call	APIM retries secondary model; if all fail → RUN_ERROR	User sees: "The AI service is temporarily unavailable"
Databricks MCP timeout	tool_executor node	Retry once (30s); if still failing → partial result with caveat	User sees: "I retrieved partial data; so results may be incomplete"
APIM token quota exceeded	APIM layer	429 response before reaching FastAPI	User sees: "You've reached your daily limit"
Postgres checkpoint failure	Any node transition	LangGraph raises CheckpointError; run fails	User sees: "Session error; please start chat"
A2A call failure (Planner)	planner_node	Retry once; if fails, orchestrator responds with clarification request	User sees: "I need more specific information to process that request"
SSE connection drop (client)	Between any two chunks	Client reconnects via fetch retry with same thread_id; LangGraph resumes from checkpoint	User sees: reconnection indicator; stream resumes from last checkpoint

11.2 Circuit Breakers

External SaaS dependencies (Oracle AP, Coupa) use the tenacity library for retry with exponential backoff and circuit breaker semantics. If a dependency exceeds the failure rate threshold, the circuit opens and subsequent calls fail fast rather than accumulating long timeouts:

```
from tenacity import retry, stop_after_attempt, wait_exponential, CircuitBreaker

@retry(stop=stop_after_attempt(3),
      wait=wait_exponential(multiplier=1, min=2, max=10))
async def call_oracle_ap(endpoint, payload, token):
    async with httpx.AsyncClient(timeout=10.0) as client:
        resp = await client.post(endpoint, json=payload,
                                headers={"Authorization": f"Bearer {token}"})
        resp.raise_for_status()
    return resp.json()
```

12. Security Summary — OAuth2 Token Flow Per Hop

This section consolidates the OAuth2/auth token that is active at each network hop in the agentic path:

Hop	Token Type	Issuer	Presented By	Validated By
Browser → App Gateway / APIM	OIDC Bearer token	Ping Identity	Browser (httpOnly cookie → Next.js → Authorization header)	APIM validate-jwt p
APIM → FastAPI (Workload 2)	Entra ID token (MI-apim)	Microsoft Entra ID	APIM Managed Identity	FastAPI trusts APIM (NetworkPolicy + ir LB)
Workload 2 → Workload 3	No token — NetworkPolicy	N/A	Pod identity via Kubernetes	Kubernetes Netwo allows only from W
Workload 3 → Content Safety	Entra ID token (MI-workload-3)	Microsoft Entra ID	DefaultAzureCredential (IMDS)	Azure AI Content S resource RBAC
Workload 3 → AI Gateway (APIM)	Entra ID token (MI-workload-3)	Microsoft Entra ID	get_bearer_token_provider (azure-identity)	APIM + Azure Oper
Workload 3 → Databricks MCP	OAuth2 SA impersonation token	Databricks OAuth2 server	SA token from Key Vault WL3	Databricks MCP + U Catalog row filter
Workload 3 → Cosmos DB	Entra ID token (MI-workload-3)	Microsoft Entra ID	DefaultAzureCredential	Cosmos DB Built-in Reader RBAC role
Workload 3 → Postgres	TLS + password	PostgreSQL	Connection string from Key Vault WL3	Postgres pg_hba.co certificate
Workload 3 → Key Vault	Entra ID token (MI-workload-3)	Microsoft Entra ID	Workload Identity OIDC token exchange	Key Vault access po scoped to MI-work

Zero standing credentials: No hop in the agentic path uses a long-lived API key, password, or connection string stored in an environment variable. Every credential either (a) comes from Key Vault at runtime via Managed Identity, or (b) is a short-lived Entra ID token generated by DefaultAzureCredential. Rotation and expiry are managed automatically — no deployment is required when a credential rotates.

13. Open Questions for Engineering Team

#	Question	Impact	Who Decides
1	Is the Semantic Query Planner a separate Kubernetes Deployment or a subgraph within the same LangGraph process?	Affects independent scaling, deployment lifecycle, and A2A vs internal function call	Architecture / ML team
2	What is the HitL spend threshold (USD) that triggers the approval gate?	Determines how often users see approval prompts; too low → friction; too high → risk	Sourcing business leadership
3	Should Content Safety be an explicit API call (current design) or an APIM-embedded policy for the AI Gateway hop?	Explicit call → independent audit trail; APIM policy → fewer hops, less visibility	Security / compliance team
4	What is the Cosmos Vector DB embedding refresh cadence and who owns the pipeline?	Stale embeddings degrade RAG tool quality; needs sprint allocation	Data engineering team
5	Is the semantic cache threshold (0.85 cosine similarity) correctly calibrated for sourcing queries?	Too high → low cache hit rate; too low → incorrect cache hits on different queries	ML team — needs eval with real query distribution
6	What is the approved model for the Semantic Query Planner — GPT-4.1 or Claude Sonnet?	Different cost, latency, and capability profile; affects planner accuracy on domain ontology	Architecture + procurement team
7	Does Kroger's hub-spoke network design require App Gateway to live in a central hub VNet rather than this application VNet?	Affects subnet layout and peering configuration	Kroger network infrastructure team